# ML Algorithms Cheat Sheet

Linear Regression · Logistic Regression · Decision Trees · SVM · KNN · Naive Bayes · Ensembles

SHEET 2 OF 4 | MACHINE LEARNING | INTERMEDIATE | PRINTABLE

## ALGORITHM QUICK-SELECTOR

When to Use What

| ALGORITHM | TASK | DATA SIZE | INTERPRETABLE? | KEY STRENGTH | KEY WEAKNESS |
|---|---|---|---|---|---|
| **Linear Regression** | Regression | Any | ✅ Yes | Fast, simple baseline | Assumes linearity |
| **Logistic Regression** | Classification | Any | ✅ Yes | Probability output, fast | Linear boundary only |
| **Decision Tree** | Both | Small–Med | ✅ Yes | No scaling needed, visual | Overfits easily |
| **Random Forest** | Both | Med–Large | ⚠ Partial | Robust, low variance | Slow on large data |
| **Gradient Boosting** | Both | Med–Large | ⚠ Partial | High accuracy, tabular SOTA | Many hyperparams, slow train |
| **SVM** | Classification | Small–Med | ❌ No | Effective in high dims | Slow on large n, kernel choice |
| **KNN** | Both | Small | ✅ Yes | No training phase | Slow at prediction, sensitive to scale |
| **Naive Bayes** | Classification | Any | ✅ Yes | Very fast, NLP-friendly | Feature independence assumption |

**Rule of thumb:** Always start with Logistic Regression (classification) or Linear Regression (regression) as a baseline before moving to complex models.

## LINEAR REGRESSION

Regression

### MODEL EQUATION

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n$$

Fit by minimizing MSE · β found via OLS or gradient descent

### OLS CLOSED-FORM SOLUTION

$$\beta = (X^T X)^{-1} X^T y$$

Works when n > p and $X^T X$ is invertible · $O(p^3)$ — use GD for large p

| ASSUMPTION | CHECK WITH |
|---|---|
| Linearity | Residual vs fitted plot |
| Independence of errors | Durbin-Watson test |
| Homoscedasticity | Scale-location plot |
| Normality of residuals | Q-Q plot, Shapiro-Wilk |
| No multicollinearity | VIF < 10 |

### SKLEARN

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train, y_train)
model.coef_       # θ₁…θₙ
model.intercept_  # θ₀
```

## LOGISTIC REGRESSION

Classification

### SIGMOID OUTPUT

$$p = \sigma(z) = 1 / (1 + e^{-z}) \text{ where } z = X\beta$$

Output is P(y=1|X) · Predict class 1 if p ≥ 0.5 (adjustable threshold)

### LOG-ODDS (LOGIT)

$$\log[p / (1-p)] = \beta_0 + \beta_1 x_1 + \ldots + \beta_n x_n$$

$e^{\beta}$ = odds ratio — how much odds multiply per unit increase in x

| VARIANT | USE WHEN |
|---|---|
| **Binary LR** | 2 classes (sigmoid output) |
| **Multinomial LR** | 3+ classes — softmax output |
| **Ordinal LR** | Ordered categories |

### SKLEARN

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression(C=1.0,  # C = 1/λ
  solver='lbfgs', max_iter=1000)
model.predict_proba(X_test)  # probabilities
```

## DECISION TREES

Both Tasks

### GINI IMPURITY (CLASSIFICATION)

$$\text{Gini} = 1 - \Sigma\, p_i^2$$

Split on feature that maximises information gain = parent impurity − weighted child impurity

### ENTROPY / INFORMATION GAIN

$$H = -\Sigma\, p_i \log_2(p_i)$$

IG = H(parent) − [weighted avg H(children)]

| HYPERPARAMETER | EFFECT |
|---|---|
| `max_depth` | Limits tree depth — prevents overfitting |
| `min_samples_split` | Min samples to split a node |
| `min_samples_leaf` | Min samples in a leaf node |
| `criterion` | gini or entropy (classification); mse (regression) |

**✅ PROS**

- No scaling needed
- Handles mixed types
- Highly interpretable

**✕ CONS**

- High variance (overfits)
- Unstable — small data changes = big tree changes

## RANDOM FOREST

Ensemble

Bagging (Bootstrap Aggregating) of many decision trees + random feature subsets at each split.

### PREDICTION

$$\hat{y} = \text{majority\_vote}(T_1(x), T_2(x), \ldots, T_n(x))$$

Classification: majority vote · Regression: mean of all tree predictions

| HYPERPARAMETER | TYPICAL VALUE | EFFECT |
|---|---|---|
| `n_estimators` | 100–500 | More trees = lower variance (diminishing returns) |
| `max_features` | √p (clf) · p/3 (reg) | Controls diversity between trees |
| `max_depth` | None (default) | Deeper trees = lower bias |
| `oob_score` | True | Free validation using out-of-bag samples |

**SKLEARN**

```
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(
  n_estimators=200, max_features='sqrt',
  oob_score=True, n_jobs=-1)
rf.feature_importances_  # Gini importance
```

## GRADIENT BOOSTING & XGBOOST

Ensemble

Sequentially fits new trees to the *residuals* of previous trees — boosting = additive model.

**ADDITIVE UPDATE**

$$F_m(x) = F_{m-1}(x) + \alpha \cdot h_m(x)$$

$\alpha$ = learning rate · $h_m$ = new tree fitted to negative gradient of loss

| HYPERPARAMETER | TYPICAL RANGE | CONTROLS |
|---|---|---|
| `n_estimators` | 100–1000 | Number of boosting rounds |
| `learning_rate` | 0.01–0.3 | Shrinkage — lower = more robust |
| `max_depth` | 3–6 | Tree complexity (shallower = better) |
| `subsample` | 0.6–0.9 | Row sampling per tree — reduces variance |
| `colsample_bytree` | 0.6–0.9 | Feature sampling per tree |
| `reg_alpha / lambda` | 0–1 | L1 / L2 regularization on leaf weights |

XGBOOST

```
import xgboost as xgb
model = xgb.XGBClassifier(
  n_estimators=300, learning_rate=0.05,
  max_depth=4, subsample=0.8,
  eval_metric='logloss',
  early_stopping_rounds=20)
```

**LightGBM vs XGBoost:** LightGBM is faster on large datasets (leaf-wise growth). XGBoost is more tuned for accuracy. Both beat vanilla GBM.

## SUPPORT VECTOR MACHINE (SVM)

Classification

**MAX-MARGIN OBJECTIVE**

$$\text{maximize } 2/\|w\| \text{ subject to } y_i(w \cdot x_i + b) \geq 1$$

Decision boundary: w·x + b = 0 · Support vectors = closest points to boundary

**SOFT MARGIN (C PARAMETER)**

$$\min \tfrac{1}{2}\|w\|^2 + C \cdot \Sigma \xi_i$$

High C = hard margin (low bias, high variance) · Low C = wide margin (high bias, low variance)

| KERNEL | FORMULA | USE WHEN |
|---|---|---|
| **Linear** | $x^T z$ | Linearly separable, high-dim (text) |
| **RBF / Gaussian** | $\exp(-\gamma\|x-z\|^2)$ | Default — non-linear, general |
| **Polynomial** | $(\gamma x^T z + r)^d$ | Image classification |
| **Sigmoid** | $\tanh(\gamma x^T z + r)$ | Neural net approximation |

**Always scale features** before SVM — RBF kernel is distance-based and sensitive to feature magnitude.

## K-NEAREST NEIGHBOURS (KNN)

Both Tasks

No training phase — prediction looks up the k closest training points and aggregates their labels.

**EUCLIDEAN DISTANCE (DEFAULT)**

$$d(x, z) = \sqrt{[\Sigma_i (x_i - z_i)^2]}$$

Also: Manhattan (L1) · Minkowski (generalises both) · Cosine for text

| K VALUE | EFFECT |
|---|---|
| **k = 1** | Low bias, very high variance — noisy decision boundary |
| **Small k** | Complex boundary — overfits |
| **Large k** | Smooth boundary — may underfit; slow prediction |
| **k = √n** | Common rule of thumb for starting point |

SKLEARN

```
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(
  n_neighbors=5, metric='euclidean',
  weights='distance')  # closer = more weight
```

**Scale features!** KNN is fully distance-based — unscaled features with large ranges dominate the distance.

## NAIVE BAYES

Classification

**BAYES' THEOREM**

$$P(y|X) \propto P(y) \cdot \Pi P(x_i|y)$$

"Naive" = assumes features are conditionally independent given class

| VARIANT | LIKELIHOOD $P(X_i|Y)$ | BEST FOR |
|---|---|---|
| **Gaussian NB** | Normal distribution | Continuous features |
| **Multinomial NB** | Multinomial distribution | Word counts, text classification |
| **Bernoulli NB** | Bernoulli distribution | Binary features (word presence) |

SKLEARN — TEXT CLASSIFICATION

```
from sklearn.naive_bayes import MultinomialNB
from sklearn.feature_extraction.text import
TfidfVectorizer
from sklearn.pipeline import Pipeline

pipe = Pipeline([
  ('tfidf', TfidfVectorizer()),
  ('clf',  MultinomialNB(alpha=1.0))
])
```

**Laplace smoothing** (alpha=1): Prevents zero probability for unseen words in test data.

## ENSEMBLE METHODS COMPARED

Ensemble

| METHOD | STRATEGY | REDUCES | TREES BUILT |
|---|---|---|---|
| **Bagging** | Parallel trees on bootstrap samples | Variance | Independently |